Beyond Answer Accuracy: Search APIs as Decision Surfaces for Tool-Using Agents

Anonymous ACL submission

Abstract

Search APIs are the fundamental retrieval layer for many agents and are often their most frequently used tool. Traditional search APIs provide URLs, titles, and snippets that preview website contents. Because full-page retrieval is token-intensive, agent retrieval architectures increasingly use progressive disclosure: the agent first sees snippets and then chooses whether to fetch full pages. In such systems, search API performance is often evaluated primarily by answer accuracy. We argue that a commercial search API is better understood as a *decision surface*: the ranked snippets, URLs, and metadata that determine whether an agent answers immediately, searches again, or spends tokens opening pages. We test this claim with one frozen GPT-5.4 agent, two tools (`search_web` and `fetch_page`), and 100 questions from SEALQA-HARD, varying only the search provider (Brave, Tavily, Firecrawl). A Kimi-K2.6 oracle labels every content element visible to the agent (URL, title, snippet, and fetched page, when fetched), producing 6,869 valid per-URL judgments. We use an audited *correct-answer* label, `semantic_match`, which preserves exact matches while accepting harmless formatting and naming variants. Under this measure, the providers remain close (25, 25, 26 / 100), but their evidence economies differ sharply: Brave offers gold-answer-rich snippets, Tavily concentrates gold-supporting URLs at rank 1, and Firecrawl encourages broad exploration. We also introduce a surface contradiction-to-gold URL ratio, which varies from 0.92 to 2.59. Provider choice is therefore a retrieval-budget and policy decision, not merely a recall decision.

1 Introduction

Early search-augmented agents often depended primarily on extracted page data provided by search API providers. As complex use cases have evolved, the agent landscape has shifted toward progressive disclosure: models receive top-n URLs, titles, snippets, and key metadata such as freshness, and then decide whether to fetch full pages. In this new landscape, engineering practice often treats search API providers as replaceable components. If two APIs return plausible top-k URLs and produce similar answer accuracy, the rest of the agent pipeline is assumed to be largely unaffected.

This paper argues that the relevant object is the *pre-fetch surface*. Before a page is opened, an agent does not see a search index or a corpus; it relies on search API results. That surface is the evidence state on which the agent decides whether to answer, search again, or spend fetch tokens. We call commercial search APIs **decision surfaces** for tool-using language agents.

For grounded language-model systems, the retrieval API is part of the grounding interface. It determines which evidence is exposed before generation, which contradictions enter context, and how much retrieval budget is spent before an answer is produced. Decision-surface evaluation therefore measures not only retrieval quality, but also the faithfulness and efficiency conditions under which grounded answers are generated.

The distinction matters because final correctness can converge while the internal pipeline diverges. A provider may expose enough snippet evidence for immediate answering. Another may put the relevant URL at rank 1, making a top-result fetch policy effective. A third may expose sparse snippets but encourage broad exploration. These regimes can yield similar aggregate accuracy with different cost, latency, contamination, and failure modes.

Under a controlled protocol, we freeze the answer model, prompt, tools, maximum iterations, judge, and page-fetch backend. The only experimental condition is the commercial search API: Brave, Tavily, or Firecrawl. We then judge ev-

Code and artifacts: https://anonymous.4open.science/r/search-api-decision-surface-99AD/

ery URL-level evidence item visible to the agent, separating pre-fetch snippet support from support discovered only after a page fetch. The per-URL oracle lets us ask not only whether the final answer was correct, but also what evidence and actions were available at the decision surface.

We do not treat provider identity as a leaderboard variable; we use Brave, Tavily, and Firecrawl as production examples of different pre-fetch evidence surfaces, showing that equal answer accuracy can hide different grounding, contradiction, and retrieval-budget regimes.

Contributions.

1. We formalize commercial search APIs as *decision surfaces* for tool-using agents rather than static ranked-list retrievers.
2. We introduce a per-URL oracle protocol that labels every API result element the agent saw, including fetched pages.
3. We distinguish pre-fetch surface support from post-fetch discovered support, and use the former to define a four-way decision partition (SMART, MISSED, BLIND, NO-OP).
4. We show that similar correctness hides different provider-induced retrieval regimes: pre-fetch support, rank concentration, blind exploration, contamination, and complementarity.

2 Related Work

Retrieval-augmented generation and open-domain QA. Retrieval-augmented generation combines parametric models with non-parametric evidence (Lewis et al., 2020; Guu et al., 2020). Open-domain QA systems such as DPR, FiD, and Atlas largely evaluate whether a retriever supplies answer-bearing passages to a reader or generator (Karpukhin et al., 2020; Izacard and Grave, 2021; Izacard et al., 2023). Our setting differs in two ways: retrieval is performed by production web-search APIs, and the reader is an agent that can choose whether to fetch pages.

Information-retrieval evaluation. Classical IR metrics, including precision/recall and rank-aware measures such as NDCG, evaluate static rankings (J"arvelin and Kek"al"ainen, 2002; Manning et al., 2008). BEIR broadened zero-shot retriever evaluation across tasks and domains (Thakur et al., 2021). Such metrics are necessary but incomplete for agents because they do not ask whether the pre-fetch surface was sufficient, misleading, or action-guiding.

Tool-using and browsing agents. WebGPT, Self-Ask, ReAct, IRCoT, Toolformer, and Self-RAG study language models that search, browse, or decide when to call tools (Nakano et al., 2021; Press et al., 2022; Yao et al., 2023; Trivedi et al., 2022; Schick et al., 2023; Asai et al., 2024). FreshLLMs shows that search augmentation helps with fresh factual knowledge and that evidence order matters (Vu et al., 2023). Over-searching work argues for cost-sensitive evaluation of search-augmented models (Xie et al., 2026). We isolate a complementary variable: the provider surface presented to the same agent.

Hard search benchmarks and evidence evaluation. GAIA, WebArena, BrowseComp, and SEALQA evaluate agents or systems on tasks requiring search, browsing, or hard factual reasoning (Mialon et al., 2024; Zhou et al., 2024; Wei et al., 2025; Pham et al., 2025). RAG evaluation frameworks such as RAGAS and ARES evaluate answer quality, faithfulness, context relevance, or generated claims (Es et al., 2024; Saad-Falcon et al., 2024). We use a hard-QA benchmark as a controlled probe for provider-induced evidence states.

Attribution, verifiability, and LLM judges. Generative search systems may produce fluent answers with incomplete citation support (Liu et al., 2023a; Yue et al., 2023); long-context work shows that more text does not guarantee robust evidence use (Liu et al., 2024). LLM-as-judge methods scale evaluation but require constrained rubrics and careful interpretation (Zheng et al., 2023; Liu et al., 2023b). Our judge labels individual retrieved documents with a fixed JSON schema, and our answer correctness label is separately audited as `semantic_match`.

Commercial search APIs. Brave, Tavily, Firecrawl, and Jina Reader expose different product surfaces (Brave Software, 2026; Tavily, 2026; Firecrawl, 2026; Jina AI, 2026). We disable provider-side page content in the main condition and use a shared `fetch_page` backend, so provider differences are attributable to ranked URLs, snippets, and metadata rather than distinct extraction pipelines.

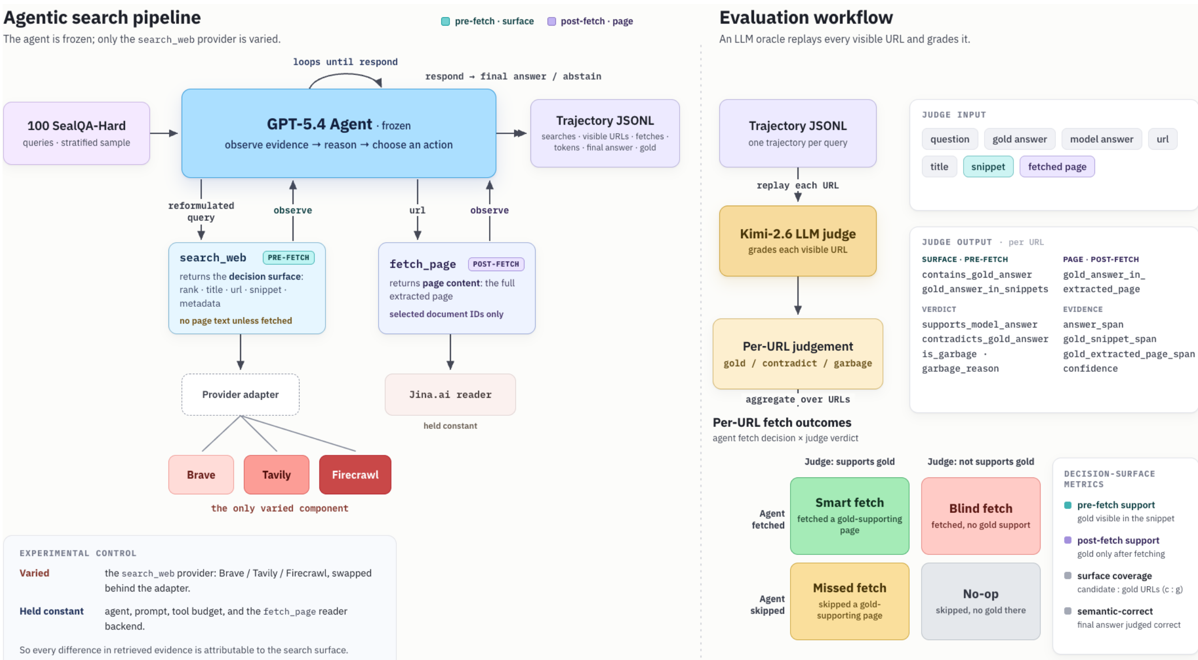

Figure 1: Execution and judging pipeline. The left lane shows the frozen agent, where only the `search_web` provider varies and `fetch_page` uses the shared Jina Reader backend. The right lane shows the oracle workflow: every visible URL is replayed to the Kimi judge in the same representation the answer model saw, yielding per-URL labels and decision-surface metrics.

3 Experimental Protocol

Figure 1 summarizes the experiment.

Dataset. We use a deterministic proportional stratified sample of 100 questions from the 254-row SEALQA-HARD subset. The strata are `freshness` × `search_results` × `topic`. We chose SEALQA-HARD because it provides challenging questions that encourage model exploration and are not yet fully resolved by a simple snippet-only search API response. Appendix B gives the full sampling summary.

Agent and tools. The answer model is GPT-5.4 with a maximum of 10 iterations. It has two tools. `search_web` accepts a query and returns up to ten ranked results, each with a document identifier, rank, title, URL, domain, snippet surface, and provider metadata. `fetch_page` accepts a document identifier from a prior search result and returns extracted markdown plus fetch metadata. The tool interface allows the agent to issue *multiple* `search_web` calls or *multiple* `fetch_page` calls in a single turn; the trace records each call separately. The final answer must use the exact prefix `FINAL ANSWER:` and be grounded in retrieved evidence.

Providers and snippet normalization. We compare Brave, Tavily, and Firecrawl Search. Each adapter normalizes provider output into the same result schema. Brave exposes additional provider-native snippet blocks. We aggregate those blocks into the generic *snippet surface* rather than treating them as a separate evidence channel; the agent saw them as snippets, and the judge saw the same text. We still discuss this asymmetry as a limitation because it changes the amount of pre-fetch text Brave returns. Tavily is run with raw content disabled, and Firecrawl is run without provider-side markdown scraping. Page text visible to the agent therefore comes only from the shared Jina Reader backend.

Run and reproducibility controls. The main provider traces were collected on 2026-05-17 UTC, from 06:49–09:02 UTC across the three providers. All providers were asked for up to 10 web results under the fixed configuration in Appendix D: Brave used US/en with moderate safe search, Tavily used basic/general search with raw content disabled, and Firecrawl used web search with provider-side markdown scraping disabled. All full-page text came from the shared Jina Reader `fetch_page` backend with a 15s timeout, 2MB read limit, concurrency 4, and gzip cache keyed by normalized URL. Because commercial search outputs are time-sensitive, the committed traces, raw provider payloads, fetch cache metadata, and judge rows are the reproducibility artifact rather than the live APIs.

Traces. The experiment produces 300 provider-query trajectories. A trace records every search call, every returned URL, every fetch decision, fetch status, token usage, latency, final answer, and gold answer. Provider-comparison scripts deterministically derive token, URL-hit, and per-query support fields from these traces. Answer correctness comes from the separate semantic audit TSV.

4 Per-URL Oracle and Metrics

Judge input. For every URL the agent saw, we run Kimi-K2.6 as the judge. The judge receives the question, gold answer, model final answer, and one retrieved document in the same XML-like format the answer model saw. For unfetched URLs, the judge sees title, URL, domain, snippet surface, and metadata. For fetched URLs, the same record also includes extracted markdown; the snippet-specific fields remain labeled separately.

Judge output. The judge returns JSON only and is instructed not to use outside knowledge. The required schema is shown below.

Field	Meaning
`contains_gold_answer`	Gold answer appears in the judged evidence.
`gold_answer_in_snippets`	Gold answer appears in provider-returned snippets.
`gold_answer_in_extracted_page`	Gold answer appears in fetched page text.
`supports_model_answer`	Evidence supports the agent's final answer.
`contradicts_gold_answer`	Evidence conflicts with the gold answer.
`is_garbage`	URL content is unusable or irrelevant.
`garbage_reason`	Short reason for a garbage label.
`answer_span`	Text span supporting the model answer.
`gold_snippet_span`	Snippet span containing gold evidence.
`gold_extracted_page_span`	Page span containing gold evidence.
`confidence`	Judge confidence score.

Across providers, 6,869 of 6,909 judge rows are valid; 40 invalid rows are excluded. Valid rows are split into 6,519 snippet-only and 350 page-visible rows.

Oracle validation. To assess whether the single-judge oracle is usable as an audit instrument, we manually validated a balanced 180-case sample of URL-label judgments. The sample covers provider × surface × label × Kimi-value slices, with five cases per leaf slice. Table 1 shows high agreement on clear human judgments overall (164/174, 94%). The remaining disagreements were boundary cases involving inferential answerability, directness of contradiction, or whether sparse and low-information results should count as unusable.

Oracle label	Audited	Clear	Agree	Agreement
`contains_gold_answer`	60	58	55	95%
`contradicts_gold_answer`	60	56	53	95%
`is_garbage`	60	60	56	93%
Overall	180	174	164	94%

Table 1: Human validation of a stratified sample of Kimi per-URL oracle labels. The sample is balanced across provider, evidence surface, label type, and Kimi positive/negative decisions. Clear excludes human judgments marked unclear.

Answer metrics. The headline CORRECT metric is `semantic_match` from a separate semantic audit TSV. The table also reports legacy token F1: we normalize the final and gold answers by lowercasing, removing articles and punctuation, and mapping simple number words to digits, then compute token-overlap F1 per query and macro-average over the 100 queries.

Support split. We separate support by when it became visible. *Pre-fetch surface support* exists for a provider-query pair when a valid judgment marks `gold_answer_in_snippets` true, or when an unfetched snippet-only judgment marks `contains_gold_answer` true. This captures support in the URL, title, snippet surface, or metadata before the agent decides whether to fetch. *Post-fetch discovered support* exists when no pre-fetch support was present, but a fetched page-visible judgment marks `gold_answer_in_extracted_page` true. *Trajectory-visible support* is the union of these two events.

Decision partition. We join the per-URL oracle with trace actions and assign each provider-query pair to one cell using pre-fetch support: SMART if pre-fetch support exists and the agent fetched a pre-fetch-supporting URL; MISSED if pre-fetch support exists but the agent fetched none of those URLs; BLIND if no pre-fetch support exists and the agent fetched at least one URL; and NO-OP if no pre-fetch support exists and no URL was fetched. The partition is descriptive, not causal: a MISSED query may still be answered correctly from snippets, and a BLIND query may still succeed if a page reveals support after the fetch.

Surface contradiction-to-gold ratio. We define the surface contradiction-to-gold ratio over snippet-only URL rows:

$$r_{c:g} = \frac{|\{u : \texttt{contradicts\_gold\_answer}(u)\}|}{|\{u : \texttt{contains\_gold\_answer}(u)\}|}.$$

Metric	Brave	Tavily	Firecrawl
CORRECT /100	25	25	26
Token F1	0.270	0.261	0.282
Avg. search calls	2.29	2.74	2.51
Avg. fetch calls	1.02	1.30	1.28
Fetched queries	65%	76%	81%
Tokens/query	59,627	54,156	57,979

Table 2: Headline metrics. CORRECT is the audited semantic match. Token F1 is normalized answer-token overlap, macro-averaged over queries; it is separate from the semantic audit. Paired-bootstrap intervals for headline differences are in Appendix J.

The metric depends on the gold answer but is independent of the model's final answer; it uses only pre-fetch surface labels, not page text.

5 Results

5.1 Correctness parity masks different evidence economies

Table 2 and Figure 2 show the headline. Under CORRECT, the providers remain close: 25, 25, and 26 correct answers out of 100. The semantic audit changes absolute counts, but not the main conclusion: aggregate correctness alone suggests the providers are nearly interchangeable.

The evidence economy says otherwise. Brave exposes pre-fetch support on 30 queries, compared with 16 for Tavily and 16 for Firecrawl. Tavily has much stronger rank concentration among pre-fetch-supporting rows: 17 rows at rank 1 (50%), versus 13% and 13% for the others. Firecrawl fetches on the largest share of queries (81%), consistent with a volume/exploration regime. We summarize paired-bootstrap uncertainty for these main contrasts after the support split in Table 4.

5.2 Pre-fetch support and page-discovered support diverge

Table 3 shows that the per-URL oracle sees a larger provider effect than headline correctness. Brave exposes substantially more pre-fetch support (30 queries) than Tavily or Firecrawl (16 and 16), while Tavily recovers more support only after fetch. The contradiction ratio also varies substantially, from 0.92 for Brave to 2.59 for Firecrawl contradicting rows per gold-supporting snippet-only row; the displayed counts make the smaller Tavily and Firecrawl denominators explicit.

Table 4 makes the intended claim boundary explicit. At this sample size, final-answer correctness differences are not meaningful: the paired intervals all include zero. The stronger result is structural

Metric	Brave	Tavily	Firecrawl
Pre-fetch support	30	16	16
Post-fetch support	3	8	3
Trajectory support	33	24	19
Snippet rows	2,095	2,339	2,085
Page-visible rows	101	125	124
Rank-1 pre-fetch	13 (13%)	17 (50%)	4 (13%)
Contradicting rows	89	58	70
Gold-supporting rows	97	31	27
$r_{c:g}$	0.92	1.87	2.59

Table 3: Support split and surface diagnostics from the per-URL judge. Pre-fetch support uses only URL/title/snippet/metadata evidence; post-fetch discovered support uses page evidence only when no pre-fetch support was present. The contradiction rows are snippet-only counts; $r_{c:g}$ divides contradicting rows by gold-supporting rows.

Claim	Contrast	Diff.	95% interval	Takeaway
Correctness parity	Brave–Tavily	0 q	[−9, 9] q	includes zero
Correctness parity	Brave–Firecrawl	−1 q	[−10, 8] q	includes zero
Pre-fetch support	Brave–Tavily	+14 q	[4, 24] q	positive
Pre-fetch support	Brave–Firecrawl	+14 q	[6, 22] q	positive
Rank-1 concentration	Tavily–Brave	+37.1 pp	[16.8, 67.3] pp	positive
Rank-1 concentration	Tavily–Firecrawl	+36.7 pp	[18.8, 66.4] pp	positive

Table 4: Main claims under paired-bootstrap uncertainty over question IDs. Differences are left minus right in native units: queries (q) or percentage points (pp). The intervals do not support ranking providers by final correctness, but they do support the evidence-economy claim: Brave exposes more pre-fetch support, while Tavily concentrates pre-fetch support at rank 1. Full intervals are in Appendix J.

rather than accuracy-based. The intervals stay positive for Brave's pre-fetch-support advantage and for Tavily's rank-1 concentration, supporting the interpretation that similar correctness hides different evidence economies.

5.3 The same agent enters different decision regimes

Figure 3 shows the mechanism. Under a strict pre-fetch definition, SMART fetches are rare for all three providers: much of the support observed in the trajectory appears only after a page is opened. Brave has the largest pre-fetch-support mass, but many such queries are MISSED rather than SMART, consistent with a snippet-rich surface where fetching is not always needed. Firecrawl's largest regime is BLIND exploration, and Tavily also shifts toward BLIND once page-discovered support is separated from the pre-fetch surface.

Decision-cell counts are visualized in Figure 3;

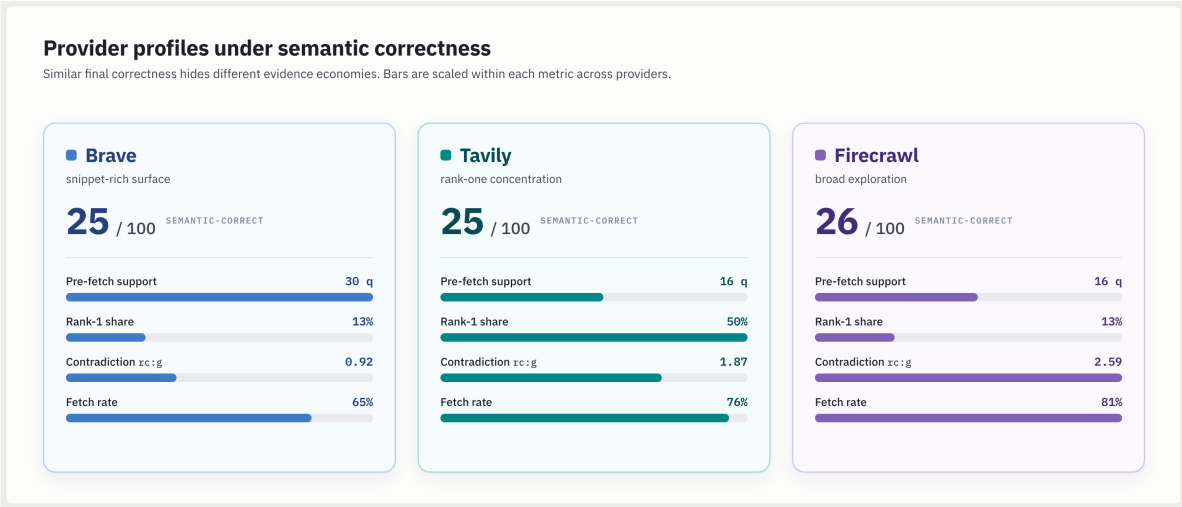

Figure 2: Provider profiles under the semantic CORRECT metric. Similar correctness hides different pre-fetch support, rank concentration, contamination, and fetch behavior. Bars are scaled within each metric across providers.

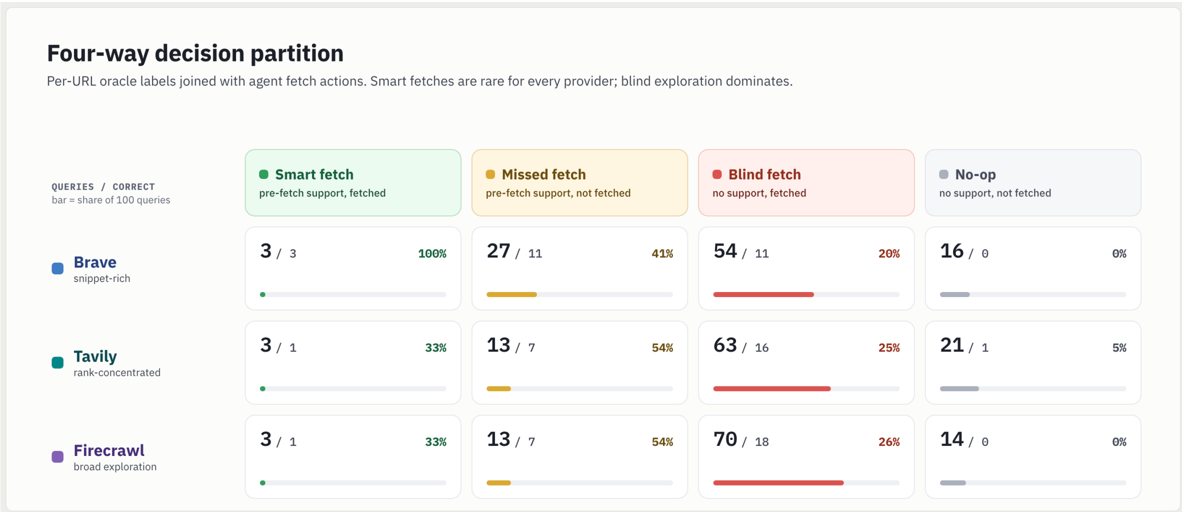

Figure 3: Four-way decision partition using pre-fetch support. Each entry is queries / correct answers (semantic match) after joining oracle labels with agent fetch actions.

Wilson intervals are reported in Appendix K, and paired-bootstrap intervals for aggregate metrics are reported in Appendix J.

5.4 Aggregate correctness hides complementarity

Only 10 questions are answered correctly by all three providers. 12 are correct under exactly two providers, 22 under exactly one, and 56 under none. Thus, provider choice changes *which* questions are solved, not only how many. This suggests that provider routing or provider-aware fetch policies may matter more than choosing a single winner.

5.5 Many failures occur despite visible answer text

The provider-comparison traces include deterministic retrieval proxies such as gold-URL hits and whether the gold answer string appears anywhere in retrieved text. These are *not* the Kimi judge or document-support labels; they are normalized string and URL checks over titles, snippets, provider-returned snippet text, and fetched page text.

Proxy	Brave	Tavily	Firecrawl
Gold URL exact hit	59	57	60
Answer in snippet surface	71	60	54
Answer in fetched page	52	57	61
Wrong despite answer text	57	56	53

Table 5: Trace-derived retrieval proxies. These deterministic checks are useful but insufficient: answer text can be present without being used correctly.

Table 5 shows why ordinary retrieval proxies are incomplete for agents. Gold URLs and answer strings often appear somewhere in the trajectory, yet the agent still has 57, 56, and 53 wrong an-

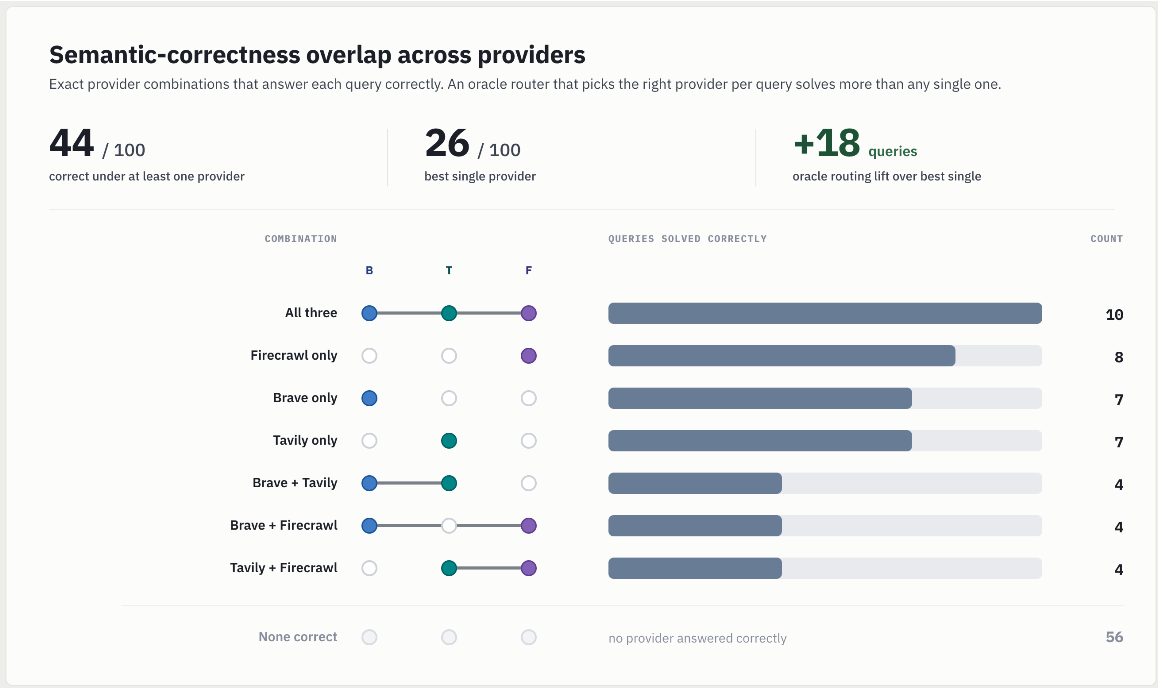

Figure 4: UpSet-style semantic-correctness overlap across providers. Rows show exact provider combinations that answer each query correctly; an oracle provider router would solve 44 queries, above the best single provider.

swers despite visible answer text. The issue is not only whether evidence exists, but whether the decision surface makes the right evidence noticeable, trusted, and actionable.

Qualitative examples. Two Brave traces show how this happens. In a Blake Shelton query, one snippet listed the five relevant 2015–2023 winning seasons, but nearby snippets repeated his nine lifetime wins; with no fetch, the agent answered *9* rather than *5*. In a border query, rank 1 favored Canada–United States, while lower snippets preserved the key word *continuous* and identified Kazakhstan–Russia; the agent again answered from the generic surface. These cases reinforce the core claim: a search surface shapes whether the model notices, trusts, fetches, and uses the right evidence amid distractors.

6 Discussion

Provider choice is a policy choice. The same agent policy has different utility under different provider surfaces. Brave's richer pre-fetch surface reduces the marginal value of fetching some support-bearing pages. Tavily's rank-one concentration suggests that, when pre-fetch support is present, top-result policies can be efficient. Firecrawl induces broader exploration and more page-time discovery. A provider should therefore be paired with a provider-aware fetch policy, not a fixed universal heuristic.

Top-k relevance is incomplete for agents. Static relevance and gold URL hit rates remain useful, but they cannot see whether the surface was already answerable, whether a fetch was worthwhile, or whether contradictory snippets competed with gold support. Decision-surface metrics complement IR metrics by measuring the action state available before the fetch decision.

What providers could optimize. An agent-ready search API should optimize not only relevance but also actionability: calibrated snippets, stable document identifiers, source/freshness metadata, low contradiction-to-gold contamination, and rankings that align answer-bearing pages with common agent fetch policies.

What practitioners can use now. The metrics here can be computed from traces without rerunning provider APIs. A practitioner can run a small sample through candidate providers, judge visible URLs, and choose a policy based on whether the provider behaves like a snippet-rich surface, a rank-concentrated surface, or a volume/exploration surface.

7 Limitations

Single agent and judge. The decision partition is induced by one GPT-5.4 agent and one Kimi-

K2.6 judge. The manual audit in Table 1 supports the oracle as a useful measurement instrument, but a different answer model, judge, prompt, or fetch appetite may shift the cell counts.

Lower-bound support. Pre-fetch support is measured only over URLs actually returned to the agent, and post-fetch support is a lower bound because unfetched URLs are not page-judged. This prevents us from claiming full provider recall.

Snippet-surface asymmetry. Brave returns more provider-native snippet text under our configuration. We aggregate it into the snippet surface to avoid presenting it as special treatment, but it remains a real product-surface difference and a plausible driver of Brave's pre-fetch support ceiling.

Observational policy analysis. The agent's actions are observed, not randomized. We do not intervene on ranks, snippets, or fetch decisions. Causal replay is future work.

Scale and provider coverage. The main experiment has 100 questions and three providers. Small cells should be read as diagnostic rather than definitive. Additional providers such as Bing, Google CSE, Serper, Exa, and others would broaden the claim.

8 Conclusion

Evaluating search APIs for tool-using agents requires looking beyond answer accuracy. In this controlled study, semantic correctness rates are close, but the internal retrieval economy changes sharply: pre-fetch support, page-time discovery, rank concentration, contradiction, fetch behavior, and instance-level correctness all depend on the provider surface. Search APIs for agents should therefore be evaluated as decision surfaces: ranked, snippet-bearing action states that allocate retrieval budget.

Ethics Statement

This work evaluates commercial search APIs and LLM agents on public web-search QA data. We do not claim that one provider is universally better. Provider names are reported because they are experimental conditions and because reproduction requires knowing which APIs produced the surfaces. The study may help reduce unnecessary page fetches and improve handling of contradictory evidence. It could also be misread as a provider leaderboard without respecting the stated limitations; we discourage that use.

Reproducibility Statement

We release the experiment code, configuration files, prompts, query-sampling logic, trace schema, evaluation scripts, bootstrap scripts, figure-generation code, paper sources, and aggregate derived artifacts needed to inspect the reported calculations. We do not publicly redistribute raw search-provider responses, rendered trace JSONLs, judge prompts or responses containing provider snippets, or fetched page text. These artifacts contain provider search results and third-party web content subject to API terms and publisher rights, and are therefore not ours to redistribute.

The reported numbers were generated from the private run artifacts using the released pipeline. Researchers can reproduce the protocol by running the same fixed query sample with their own API credentials and then applying the released evaluation scripts. Because commercial search APIs are live, time-varying systems, reruns reproduce the experimental procedure rather than bit-identical provider surfaces. To support auditability without redistributing raw provider content, we release aggregate summaries, row counts, semantic-audit labels, validation summaries, and deterministic scripts that map traces and judge rows to every reported table and figure.

References

Akari Asai, Zeqiu Wu, Yizhong Wang, Avirup Sil, and Hannaneh Hajishirzi. 2024. Self-RAG: Learning to retrieve, generate, and critique through self-reflection. In *International Conference on Learning Representations*.

Brave Software. 2026. Brave search api reference. https://api-dashboard.search.brave.com/api-reference/web/search/get.

Shahul Es, Jithin James, Luis Espinosa-Anke, and Steven Schockaert. 2024. RAGAS: Automated evaluation of retrieval augmented generation. In *Proceedings of the 18th Conference of the European Chapter of the Association for Computational Linguistics: System Demonstrations*.

Firecrawl. 2026. Firecrawl search api documentation. https://docs.firecrawl.dev/api-reference/endpoint/search.

Kelvin Guu, Kenton Lee, Zora Tung, Panupong Pasupat, and Ming-Wei Chang. 2020. REALM: Retrieval-augmented language model pre-training. In *Proceedings of the 37th International Conference on Machine Learning*.

Gautier Izacard and Edouard Grave. 2021. Leveraging passage retrieval with generative models for open domain question answering. In *Proceedings of the 16th Conference of the European Chapter of the Association for Computational Linguistics*, pages 874–880.

Gautier Izacard, Patrick Lewis, Maria Lomeli, Lucas Hosseini, Fabio Petroni, Timo Schick, Jane Dwivedi-Yu, Armand Joulin, Sebastian Riedel, and Edouard Grave. 2023. ATLAS: Few-shot learning with retrieval augmented language models. *Journal of Machine Learning Research*, 24(251):1–43.

Kalervo J"arvelin and Jaana Kek"al"ainen. 2002. Cumulated gain-based evaluation of ir techniques. *ACM Transactions on Information Systems*, 20(4):422–446.

Jina AI. 2026. Jina reader: Url to llm-friendly input. https://r.jina.ai/.

Vladimir Karpukhin, Barlas Oguz, Sewon Min, Patrick Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, and Wen-tau Yih. 2020. Dense passage retrieval for open-domain question answering. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing*.

Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Kuttler, Mike Lewis, Wen-tau Yih, Tim Rocktaschel, Sebastian Riedel, and Douwe Kiela. 2020. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems*, volume 33.

Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. 2024. Lost in the middle: How language models use long contexts. *Transactions of the Association for Computational Linguistics*, 12:157–173.

Nelson F. Liu, Tianyi Zhang, and Percy Liang. 2023a. Evaluating verifiability in generative search engines. *arXiv preprint arXiv:2304.09848*.

Yang Liu, Dan Iter, Yichong Xu, Shuohang Wang, Ruochen Xu, and Chenguang Zhu. 2023b. G-Eval: NLG evaluation using GPT-4 with better human alignment. *arXiv preprint arXiv:2303.16634*.

Christopher D. Manning, Prabhakar Raghavan, and Hinrich Sch"utze. 2008. *Introduction to Information Retrieval*. Cambridge University Press.

Gregoire Mialon, Clementine Fourrier, Craig Swift, Thomas Wolf, Yann LeCun, and Thomas Scialom. 2024. GAIA: A benchmark for general ai assistants. In *International Conference on Learning Representations*.

Reiichiro Nakano, Jacob Hilton, Suchir Balaji, Jeff Wu, Long Ouyang, Christina Kim, Christopher Hesse, Shantanu Jain, Vineet Kosaraju, William Saunders, Xu Jiang, Karl Cobbe, Tyna Eloundou, Gretchen Krueger, Kevin Button, Matthew Knight, Benjamin Chess, and John Schulman. 2021. WebGPT: Browser-assisted question-answering with human feedback. *arXiv preprint arXiv:2112.09332*.

Thinh Pham, Nguyen Nguyen, Pratibha Zunjare, Weiyuan Chen, Yu-Min Tseng, and Tu Vu. 2025. SealQA: Raising the bar for reasoning in search-augmented language models. *arXiv preprint arXiv:2506.01062*.

Ofir Press, Muru Zhang, Sewon Min, Ludwig Schmidt, Noah A. Smith, and Mike Lewis. 2022. Measuring and narrowing the compositionality gap in language models. *arXiv preprint arXiv:2210.03350*.

Jon Saad-Falcon, Omar Khattab, Christopher Potts, and Matei Zaharia. 2024. ARES: An automated evaluation framework for retrieval-augmented generation systems. *arXiv preprint arXiv:2311.09476*.

Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. 2023. Toolformer: Language models can teach themselves to use tools. In *Advances in Neural Information Processing Systems*.

Tavily. 2026. Tavily search api documentation. https://docs.tavily.com/documentation/api-reference/endpoint/search.

Nandan Thakur, Nils Reimers, Andreas Rucklé, Abhishek Srivastava, and Iryna Gurevych. 2021. BEIR: A heterogeneous benchmark for zero-shot evaluation of information retrieval models. In *Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks*.

Harsh Trivedi, Niranjan Balasubramanian, Tushar Khot, and Ashish Sabharwal. 2022. Interleaving retrieval with chain-of-thought reasoning for knowledge-intensive multi-step questions. *arXiv preprint arXiv:2212.10509*.

Tu Vu, Mohit Iyyer, Xuezhi Wang, Noah Constant, Jerry Wei, Jason Wei, Chris Tar, Yun-Hsuan Sung, Denny Zhou, Quoc Le, and Thang Luong. 2023. FreshLLMs: Refreshing large language models with search engine augmentation. *arXiv preprint arXiv:2310.03214*.

Jason Wei, Zhiqing Sun, Spencer Papay, Scott McKinney, Jeffrey Han, Isa Fulford, Hyung Won Chung, Alex Tachard Passos, William Fedus, and Amelia Glaese. 2025. BrowseComp: A simple yet challenging benchmark for browsing agents. *arXiv preprint arXiv:2504.12516*.

Roy Xie, Deepak Gopinath, David Qiu, Dong Lin, Haitian Sun, Saloni Potdar, and Bhuwan Dhingra. 2026. Over-searching in search-augmented large language models. *arXiv preprint arXiv:2601.05503*.

Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. 2023. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations*.

Xiang Yue, Boshi Wang, Ziru Chen, Kai Zhang, Yu Su, and Huan Sun. 2023. Automatic evaluation of attribution by large language models. *arXiv preprint arXiv:2305.06311*.

Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric P. Xing, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica. 2023. Judging LLM-as-a-judge with MT-bench and chatbot arena. *arXiv preprint arXiv:2306.05685*.

Shuyan Zhou, Frank F. Xu, Hao Zhu, Xuhui Zhou, Robert Lo, Abishek Sridhar, Xianyi Cheng, Tianyue Ou, Yonatan Bisk, Daniel Fried, Uri Alon, and Graham Neubig. 2024. WebArena: A realistic web environment for building autonomous agents. In *International Conference on Learning Representations*.

A Formal Metric Definitions

For a provider p and query q, let $U_{p,q}$ denote the valid judged URLs visible in the trajectory. Unfetched URLs have snippet-only judgments; fetched URLs have page-visible judgments that include both the pre-fetch snippet fields and the fetched page block. Let $G_{\mathrm{pre}}(u)$ be true when the pre-fetch surface supports the gold answer: either `gold_answer_in_snippets` is true, or a snippet-only judgment marks `contains_gold_answer` true. Let $G_{\mathrm{page}}(u)$ be true when a page-visible judgment marks `gold_answer_in_extracted_page` true. Let $F(u)$ be true when the trace contains a `fetch_page` call for u.

For each provider-query pair, pre-fetch support is $\exists u : G_{\mathrm{pre}}(u)$. Post-fetch discovered support is $\neg\exists u : G_{\mathrm{pre}}(u)$ and $\exists u : G_{\mathrm{page}}(u)$. Trajectory-visible support is their union.

Cell	Condition	Interpretation
SMART	$\exists u : G_{\mathrm{pre}}(u)$ and $\exists u : G_{\mathrm{pre}}(u) \wedge F(u)$	The pre-fetch surface exposed support and the agent opened it.
MISSED	$\exists u : G_{\mathrm{pre}}(u)$ and $\neg\exists u : G_{\mathrm{pre}}(u) \wedge F(u)$	Pre-fetch support was visible but not fetched.
BLIND	$\neg\exists u : G_{\mathrm{pre}}(u)$ and $\exists u : F(u)$	The agent spent fetch budget without pre-fetch support.
NO-OP	$\neg\exists u : G_{\mathrm{pre}}(u)$ and $\neg\exists u : F(u)$	No pre-fetch support and no fetch.

The partition is computed per provider-query pair after joining judge rows to trace actions by normalized URL and retrieval provenance. It is descriptive: a MISSED query can still be answered correctly from snippets, and a BLIND query can still succeed if a fetched page reveals evidence not present in snippets. The surface contradiction-to-gold ratio is computed only over snippet-only rows:

$$r_{c:g} = \frac{|\{u : \texttt{contradicts\_gold\_answer}(u)\}|}{|\{u : \texttt{contains\_gold\_answer}(u)\}|}.$$

It is independent of the model's final answer, but not independent of the gold answer.

For answer-level scoring, exact match and token F1 use the same deterministic normalizer: lowercasing, removing articles and punctuation, and mapping simple number words to digits. Token F1 is the token-overlap F1 between the normalized final answer and normalized gold answer, macro-averaged over the 100 queries. The semantic CORRECT metric is separate and comes from the audited `semantic_match` TSV.

B Dataset Stratification

The sample file records dataset `vtllms/sealqa`, subset `seal_hard`, sample size 100, source size 254, seed 20260509, and strata `freshness`, `search_results`, and `topic`. Marginal counts are: 25 fast-changing, 43 slow-changing, 32 never-changing; 57 conflicting and 43 unhelpful search-result labels; topics are Science & Technology 27, Sports 22, Entertainment 22, Others 12, Politics 9, and History & Geography 8. Effective years are 61 before 2024, 16 in 2024, and 23 in 2025. Multi-label question-type tags include 72 advanced reasoning, 61 entity/event disambiguation, 13 temporal tracking, 5 cross-lingual, and 2 false-premise.

C Fixed Agent Contract

The answer model, prompt, tools, and loop budget are fixed across providers. The agent receives a factual question in <user_query> tags and can call `search_web` and, in the main condition, `fetch_page`.

Component	Fixed value
Answer model	GPT-5.4 Azure deployment, temperature 0
Loop budget	10 iterations, up to 10 results per search call
Tools	`search_web` over one configured provider; `fetch_page` using model-visible `document_id`
Turn structure	The model may issue multiple search or fetch calls in one turn; each call is traced separately
Search observation	`document_id`, rank, title, URL, domain, snippet surface, provider metadata
Fetch observation	Source provenance, fetch metadata, and extracted markdown/text from the shared backend
Final answer rule	Response must begin with `FINAL ANSWER:` and contain only the short factual answer
Hidden fields	Gold answers and gold URLs are retained in traces for offline grading, never sent to the answer model

In fetch-tool mode, `search_web` observations are snippet-only. Page text enters the model context only after the model chooses a prior `document_id`. This makes fetch behavior observable and prevents provider comparisons from being driven by automatic page inclusion.

D Provider Request Configurations

All providers are configured to return web-search results rather than provider-side page extracts. Each adapter normalizes provider output to the same model-visible schema before rendering.

Provider	Request settings	Snippet field	Disabled content
Brave	GET, count 10, offset 0, US/en, moderate safe search, web-only	`description` plus `extra_snippets`	Text decorations
Tavily	POST, `search_depth=basic`, `topic=general`, max 10	`content`	Answer, images, favicon, raw content
Firecrawl	v2 search, limit 10, web source only, US, 60s timeout	`description` / metadata description	Provider-side markdown scraping

The normalized result fields are `rank`, `title`, `url`, `domain`, `snippet`, and `provider_metadata`. URL normalization removes fragments and common tracking parameters. Brave's additional snippet blocks are rendered as <extra_snippet> elements because they were visible to the agent as part of the provider surface; we do not reclassify them as fetched-page evidence.

E Shared Page Fetch Backend

All page text in the main experiment comes from the same `fetch_page` backend, independent of the search provider. The backend is Jina Reader markdown through `r.jina.ai`; it is invoked only after the agent selects a `document_id`. Fetch artifacts are cached as gzip JSON records keyed by the normalized URL and backend, with fetch status, final URL, content type, truncation flag, extractor name, extracted character count, token estimate, text hash, latency, and extracted text. The configured guardrails are a 15s timeout, a 2MB maximum read, and concurrency 4. In the analyzed run, successful fetches are recorded as `jina_reader_markdown`; failures are retained as failed fetches rather than silently dropped.

F Judge Prompt and Output Schema

The Kimi-K2.6 judge is run one URL at a time. The judge receives the question, gold answer, model final answer, and one retrieved document rendered in the same XML-like representation that the answer model saw. Snippet-only records contain title, URL, domain, snippets, extra snippets, and metadata. Page-visible records additionally contain the fetched page block that was returned by `fetch_page`.

Judge guardrail	**Implementation**
No external knowledge	Prompt restricts the judge to supplied document content
Strict output	JSON-only schema with support, contradiction, garbage, spans, and confidence
Valid-row filter	Requires schema version, parsed judgment, provider/query/retrieval/URL IDs, and no execution or parse error
Garbage handling	Deterministic precheck flags failed, unsupported, empty, or media fetches before LLM judging
Duplicate control	Exact prompt-cache reuse; optional query-URL reuse separates snippet-only from page-visible surfaces
Error policy	Invalid judge rows are excluded, not converted into negative evidence

The schema fields used in the paper are `contains_gold_answer`, `gold_answer_in_snippets`, `gold_answer_in_extracted_page`, `supports_model_answer`, `contradicts_gold_answer`, `is_garbage`, evidence spans, and confidence. Across the main artifacts, 6,869 of 6,909 rows are valid, split into 6,519 snippet-only and 350 page-visible rows.

G Example Judgments and Semantic Corrections

The semantic audit marks some EM misses as correct when the answer is semantically equivalent. Representative examples are: Brave *Astra Zeneca* vs. *AstraZeneca*; Brave *UnionPay* vs. *China UnionPay*; Tavily *3 players* vs. *3*; Firecrawl *Bohemian Rhapsody* vs. *Bohemian Rhapsody, 9,948,386 viewers*; and Firecrawl *16 years* vs. *16 years old*. The regenerated audit file records the exact TSV rows and judge notes. Per-URL judge examples can be inspected by joining the judge JSONLs with the trace viewer: support-bearing rows have `contains_gold_answer` true and a span; contamination rows have `contradicts_gold_answer` true; relevant-but-insufficient rows remain false for both support fields.

H Trace Schema and Reproducibility

Each provider-query run is one JSONL trace with the full sequence of LLM request snapshots, tool calls, normalized search responses, fetch records, token usage, latency, final response, extracted final answer, and offline gold fields. The trace schema supports recomputing metrics without rerunning provider APIs. Raw provider payloads are preserved under `raw_response`; rendered tool observations are preserved inside the request snapshots, which allows later audits of exactly what the answer model saw.

The figure script regenerates all paper macros and figures from four artifact families: the semantic audit TSV, the three trace JSONLs, the three Kimi judge JSONLs, and provider-comparison summaries. Strict make targets fail if raw judge files are missing or still Git LFS pointers. The test suite covers several protocol invariants: fetch-tool mode keeps search observations snippet-only, `fetch_page` resolves prior document IDs to URLs internally, judge prompts include the expected document fields and schema, unsupported page types are marked unusable, and provider summaries count recovered transient failures.

I Trace Viewer and Qualitative Audit

The repository includes human-readable trajectory views in `results/trace_views/`. The rendering script is `scripts/render_trace.py`, and `scripts/trace_dashboard.py` provides a local browsing dashboard. These views were used for spot-checking case studies, verifying multi-call turns, and inspecting failures where answer text was present but unused. They are not an independent metric source: the paper numbers come from trace JSONLs, judge JSONLs, the semantic TSV, and provider-comparison summaries.

J Paired Bootstrap Uncertainty

We resample question IDs with replacement, preserving the matched Brave/Tavily/Firecrawl rows for each question. The tables use 10,000 percentile-bootstrap replicates with seed 20260628. Count metrics are reported per 100 sampled questions; pairwise differences are left minus right in the row's native units.

Metric	Brave	Tavily	Firecrawl
CORRECT /100	25 [17, 34]	25 [17, 34]	26 [18, 35]
Pre-fetch support /100	30 [21, 39]	16 [9, 23]	16 [9, 23]
Rank-1 pre-fetch	12.9 [7.1, 19.4]	50.0 [31.2, 78.9]	13.3 [4.2, 26.3]
$r_{c:g}$	0.92 [0.49, 1.73]	1.87 [0.83, 4.47]	2.59 [1.11, 8.00]
Fetched queries /100	65 [56, 74]	76 [67, 84]	81 [73, 89]
Avg. fetch calls	1.02 [0.81, 1.25]	1.30 [1.06, 1.55]	1.28 [1.08, 1.50]
Tokens/query	59,627 [47,706, 72,908]	54,156 [44,304, 64,991]	57,979 [44,635, 72,878]

Table 6: Provider-level paired-bootstrap 95% intervals.

Metric	Brave–Tavily	Brave–Firecrawl	Tavily–Firecrawl
CORRECT /100	0 [−9, 9]	−1 [−10, 8]	−1 [−10, 8]
Pre-fetch support /100	14 [4, 24]	14 [6, 22]	0 [−8, 8]
Rank-1 pre-fetch	−37.1 [−67.3, −16.8]	−0.5 [−13.3, 10.1]	36.7 [18.8, 66.4]
$r_{c:g}$	−0.95 [−3.22, 0.00]	−1.68 [−6.78, −0.36]	−0.72 [−4.86, 0.53]
Fetched queries /100	−11 [−19, −3]	−16 [−25, −7]	−5 [−12, 1]
Avg. fetch calls	−0.28 [−0.55, −0.02]	−0.26 [−0.51, −0.02]	0.02 [−0.23, 0.27]
Tokens/query	5,471 [−4,591, 16,176]	1,648 [−11,834, 13,959]	−3,823 [−18,019, 9,064]

Table 7: Paired provider differences with bootstrap 95% intervals. Differences are left minus right.

K Additional Diagnostics and Artifact Manifest

The tables below report selected diagnostics that support the main claims without changing the headline conclusions.

Metric	Brave	Tavily	Firecrawl
Gold URL exact hit	59	57	60
Gold domain hit	82	82	82
Gold source-family hit	61	60	64
Answer in snippet surface	71	60	54
Answer in fetched page	52	57	61
Answer in any retrieved text	78	75	76
Wrong despite answer text	57	56	53

Table 8: Deterministic retrieval diagnostics from trace-derived provider summaries. These string and URL proxies are separate from the Kimi per-URL support labels.

Provider/cell	Correct rate and Wilson 95% CI
Brave SMART	3/3: 100% [44–100]
Brave MISSED	11/27: 41% [25–59]
Brave BLIND	11/54: 20% [12–33]
Brave NO-OP	0/16: 0% [0–19]
Tavily SMART	1/3: 33% [6–79]
Tavily MISSED	7/13: 54% [29–77]
Tavily BLIND	16/63: 25% [16–37]
Tavily NO-OP	1/21: 5% [1–23]
Firecrawl SMART	1/3: 33% [6–79]
Firecrawl MISSED	7/13: 54% [29–77]
Firecrawl BLIND	18/70: 26% [17–37]
Firecrawl NO-OP	0/14: 0% [0–22]

Table 9: Wilson 95% intervals for decision-cell correctness under the pre-fetch decision partition.

Pair	both	left only	right only	both wrong
Brave vs. Firecrawl	14	11	12	63
Brave vs. Tavily	14	11	11	64
Firecrawl vs. Tavily	14	12	11	63

Table 10: Pairwise semantic-correct overlap derived from the semantic audit TSV.

Metric	Brave	Tavily	Firecrawl
Total tokens	5.96M	5.42M	5.80M
Median/query	40,638	36,867	34,383
Max/query	303,462	305,474	380,646
>100k queries	19	16	16
Fetch success/fail	92/10	119/11	121/7

Table 11: Token and fetch diagnostics from provider summaries.

Raw artifacts. Audit mode consumes the semantic audit TSV, three phase-1 trace JSONLs, three Kimi per-URL judge JSONLs, and the provider-comparison summary artifacts. The script checks for Git LFS pointer files and fails explicitly if raw data has not been pulled.